DIGITAL VERIFICATION DIPLOMA

Assignment 6 extra

NAME:	Roaa Atef Mohamed

1. ALSU DESIGN:

```
import shared_pkg::*;

module ALSU(alsu_if.DUT a_if);
reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
reg  [1:0] cin_reg;
reg [2:0] opcode_reg;
reg signed [2:0] A_reg, B_reg;
reg  signed [5:0] shift_out_reg;
wire invalid_red_op, invalid_opcode, invalid;
//Invalid handling
assign invalid_red_op = (red_op_A_reg || red_op_B_reg) & (opcode_reg[1] || opcode_reg[2]);
assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
assign invalid = invalid_red_op || invalid_opcode;
//Registering input signals
always @(posedge a_if.clk or posedge a_if.rst) begin
  if(a_if.rst) begin
     cin_reg <= 0;
     red_op_B_reg <= 0;
     red_op_A_reg <= 0;
     bypass_B_reg <= 0;
     bypass_A_reg <= 0;
     direction_reg <= 0;
     serial_in_reg <= 0;
     opcode_reg <= 0;
     A_reg <= 0;
     B_reg <= 0;
  end else begin
     cin_reg <= a_if.cin;
     red_op_B_reg <= a_if.red_op_B;
     red_op_A_reg <= a_if.red_op_A;
     bypass_B_reg <= a_if.bypass_B;
     bypass_A_reg <= a_if.bypass_A;
     direction_reg <= a_if.direction;
     serial_in_reg <= a_if.serial_in;
     opcode_reg <= a_if.opcode;
     A_reg <= a_if.A;
     B_reg <= a_if.B;
  end
end

//leds output blinking
always @(posedge a_if.clk or posedge a_if.rst) begin
  if(a_if.rst) begin
     a_if.leds <= 0;
  end else begin
      if (invalid)
        a_if.leds <= ~a_if.leds;
      else
        a_if.leds <= 0;
  end
end

//ALSU output processing
always @(posedge a_if.clk or posedge a_if.rst) begin
  if(a_if.rst) begin
```

```
        a_if.out <= 0;
      end
      else begin
        if (bypass_A_reg && bypass_B_reg)
          a_if.out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
        else if (bypass_A_reg)
          a_if.out <= A_reg;
        else if (bypass_B_reg)
          a_if.out <= B_reg;
        else if (invalid)
            a_if.out <= 0;
        else begin
            case ( opcode_reg)
              3'h0: begin
                if (red_op_A_reg && red_op_B_reg)
                  a_if.out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
                else if (red_op_A_reg)
                  a_if.out <= |A_reg;
                else if (red_op_B_reg)
                  a_if.out <= |B_reg;
                else
                  a_if.out <= A_reg | B_reg;
              end
              3'h1: begin
                if (red_op_A_reg && red_op_B_reg)
                  a_if.out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
                else if (red_op_A_reg)
                  a_if.out <= ^A_reg;
                else if (red_op_B_reg)
                  a_if.out <= ^B_reg;
                else
                  a_if.out <= A_reg ^ B_reg;
              end
              3'h2: begin
                if(FULL_ADDER=="ON")
              a_if.out <= A_reg + B_reg+cin_reg;
            else
               a_if.out <= A_reg + B_reg;
           end
              3'h3: a_if.out <= A_reg * B_reg;
              3'h4: begin

                a_if.out<=shift_out_reg;
              end
              3'h5: begin

                   a_if.out<=shift_out_reg;
              end
            endcase
        end
      end
    end

    endmodule

```

2. alsu agent:

```
package alsu_agent_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import alsu_sequencer_pkg::*;
    import alsu_monitor_pkg::*;
    import alsu_seq_item_pkg::*;
    import alsu_driver_pkg::*;
    import alsu_config_pkg::*;

    class alsu_agent extends uvm_agent;
        `uvm_component_utils(alsu_agent)
        alsu_sequencer sqr;
        alsu_driver driver;
        alsu_monitor monitor;
        alsu_config alsu_cfg;
        uvm_analysis_port #(alsu_seq_item) agent_ap;

        function new (string name = "shift_reg_agent", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            alsu_cfg = alsu_config::type_id::create("alsu_cfg");
            agent_ap = new("agent_ap", this);
            if (!uvm_config_db #(alsu_config)::get(this , "", "CFG", alsu_cfg))begin
                `uvm_fatal("build_phase" , "Unable to get configuration object")
            end
           if(alsu_cfg.is_active==UVM_ACTIVE) begin
            sqr = alsu_sequencer::type_id::create("sqr",this);
            driver = alsu_driver::type_id::create("driver",this);
            monitor = alsu_monitor::type_id::create("monitor",this);
           end
           else begin
            monitor = alsu_monitor::type_id::create("monitor",this);
           end

        endfunction

        function void connect_phase(uvm_phase phase);
            if(alsu_cfg.is_active==UVM_ACTIVE) begin
            driver.alsu_vif = alsu_cfg.alsu_vif;
            monitor.alsu_vif = alsu_cfg.alsu_vif;
            driver.seq_item_port.connect(sqr.seq_item_export);
            monitor.monitor_ap.connect(agent_ap);
        end
        else begin
            monitor.alsu_vif = alsu_cfg.alsu_vif;
            monitor.monitor_ap.connect(agent_ap);
        end
        endfunction
    endclass

endpackage
```

3. alsu config :

```systemverilog
package alsu_config_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

    class alsu_config extends uvm_object;
        `uvm_object_utils(alsu_config)

        virtual alsu_if alsu_vif;
      uvm_active_passive_enum is_active;
        function new(string name = "alsu_config");
            super.new(name);
        endfunction
    endclass
endpackage
```

4. alsu coverage :

```
package alsu_coverage_pkg;
    import uvm_pkg::*;
    import alsu_seq_item_pkg::*;
    import shared_pkg::*;
    `include "uvm_macros.svh"
    class alsu_coverage extends uvm_component;
        `uvm_component_utils(alsu_coverage)
        uvm_analysis_export #(alsu_seq_item) coverage_export;
        uvm_tlm_analysis_fifo #(alsu_seq_item) cov_fifo;
        alsu_seq_item seq_item_cov;
        /*............................cover group............................*/
        covergroup alsu_cg ;
    /*................coverpoint for a maxpos,maxneg,0 values.................*/
    A_cp1: coverpoint seq_item_cov.A
    {
    bins A_data_0 = {0};
    bins A_data_max = {MAXPOS};
    bins A_data_min = {MAXNEG};
    bins A_data_default=default;
    }
    /*................coverpoint for when takes walkingone.................*/
    A_cp2: coverpoint seq_item_cov.A iff(seq_item_cov.red_op_a)
    {
bins walkingones []={3'b001,3'b010,3'b100};
    }
    /*................coverpoint for b maxpos,maxneg,0 values.................*/
    B_cp1: coverpoint seq_item_cov.B
    {
    bins B_data_0 = {0};
    bins B_data_max = {MAXPOS};
    bins B_data_min = {MAXNEG};
    bins B_data_default=default;
    }
        /*................coverpoint for when takes walkingone.................*/
    B_cp2: coverpoint seq_item_cov.B iff(!seq_item_cov.red_op_a&seq_item_cov.red_op_b)
    {
    bins walkingones []={3'b001,3'b010,3'b100};
    }
    ///////////////////////***opcode coverpoint***////////////////////
    ALU_cp:coverpoint seq_item_cov.op_code
    {
    bins Bins_shift[] = {SHIFT,ROTATE};
    bins Bins_arith [] = {ADD,MULT};
    bins Bins_bitwise [] = {OR,XOR};
    bins Bins_invalid ={INVALID_6,INVALID_7};
    bins Bins_trans = (OR=>XOR=>ADD=>MULT=>SHIFT=>ROTATE);
    }
    c_cp:coverpoint seq_item_cov.cin
    {
    bins one = {1};
    bins zero = {0};
    }

    direction_cp:coverpoint seq_item_cov.direction
    {
```

```
        bins one = {1};
        bins zero = {0};
        }
            serial_cp:coverpoint seq_item_cov.serial_in
        {
        bins one = {1};
        bins zero = {0};
        }
            reda_cp:coverpoint seq_item_cov.red_op_a
        {
        bins one = { 1};
        bins zero = {0};
        }
            redb_cp:coverpoint seq_item_cov.red_op_b
        {
        bins one = {1};
        bins zero = {0};
        }
     c1_cs : cross B_cp1,ALU_cp
    {
    bins add_mult_b1 = binsof(ALU_cp.Bins_arith) && binsof(B_cp1.B_data_max);
        bins add_mult_b2= binsof(ALU_cp.Bins_arith)&& binsof(B_cp1.B_data_0);
        bins add_mult_b3 = binsof(ALU_cp.Bins_arith)&&binsof(B_cp1.B_data_min);
    option.cross_auto_bin_max=0;
    }
     c2_cs : cross A_cp1,ALU_cp
    {
    bins add_mult_a1 = binsof(ALU_cp.Bins_arith) && binsof(A_cp1.A_data_max);
        bins add_mult_a2= binsof(ALU_cp.Bins_arith)&& binsof(A_cp1.A_data_0);
        bins add_mult_a3 = binsof(ALU_cp.Bins_arith)&&binsof(A_cp1.A_data_min);
    option.cross_auto_bin_max=0;
    }
    c3_cs : cross ALU_cp,c_cp
    {
        bins add = binsof(ALU_cp) intersect {ADD} && (binsof(c_cp.zero)||binsof(c_cp.one));
        option.cross_auto_bin_max=0;
    }
    c4_cs : cross  ALU_cp,direction_cp
    {
        bins sh = binsof (ALU_cp) intersect {SHIFT} && (binsof (direction_cp.zero)||binsof (direction_cp.one));
        bins r = binsof (ALU_cp) intersect {ROTATE} && (binsof (direction_cp.zero)||binsof (direction_cp.one));
        option.cross_auto_bin_max=0;
    }
    c5_cs : cross  ALU_cp,serial_cp
    {
        bins sh = binsof (ALU_cp) intersect {SHIFT} && (binsof (serial_cp.zero)||binsof (serial_cp.one));
        option.cross_auto_bin_max=0;
    }
    c6_cs : cross  ALU_cp,reda_cp
    {
        bins reda = binsof (ALU_cp.Bins_bitwise) && (binsof (reda_cp.one));
        option.cross_auto_bin_max=0;
    }
    c7_cs : cross  ALU_cp,redb_cp
    {
```

```
c5_cs : cross  ALU_cp,serial_cp
{
    bins sh = binsof (ALU_cp) intersect {SHIFT} && (binsof (serial_cp.zero)||binsof (serial_cp.one));
    option.cross_auto_bin_max=0;
}
c6_cs : cross  ALU_cp,reda_cp
{
    bins reda = binsof (ALU_cp.Bins_bitwise) && (binsof (reda_cp.one));
    option.cross_auto_bin_max=0;
}
c7_cs : cross  ALU_cp,redb_cp
{
    bins redb = binsof (ALU_cp.Bins_bitwise) && (binsof (redb_cp.one));
    option.cross_auto_bin_max=0;
}
c8_cs : cross ALU_cp,redb_cp,reda_cp
{
bins a1 = binsof(ALU_cp.Bins_shift) &&(binsof(reda_cp.one))&&(binsof(redb_cp.zero));
bins a2 = binsof(ALU_cp.Bins_arith) &&(binsof(reda_cp.one))&&(binsof(redb_cp.zero));
bins b1 = binsof(ALU_cp.Bins_shift) &&(binsof(redb_cp.one))&&(binsof(reda_cp.zero));
bins b2 = binsof(ALU_cp.Bins_arith) &&(binsof(redb_cp.one))&&(binsof(reda_cp.zero));
bins ab1 = binsof(ALU_cp.Bins_shift) &&(binsof(redb_cp.one))&&(binsof(reda_cp.one));
bins ab2 = binsof(ALU_cp.Bins_arith) &&(binsof(redb_cp.one))&&(binsof(reda_cp.one));
option.cross_auto_bin_max=0;
}
endgroup
/*............................................*/
        function new(string name = "alsu_coverage", uvm_component parent = null) ;
            super.new(name, parent);
            alsu_cg = new();
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            coverage_export = new("coverage_export", this);
            cov_fifo = new("cov_fifo", this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            coverage_export.connect(cov_fifo.analysis_export);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                cov_fifo.get(seq_item_cov);
                 if (!(seq_item_cov.rst||seq_item_cov.bypass_a||seq_item_cov.bypass_b)) begin
               alsu_cg.sample();
            end
            end
        endtask
    endclass
endpackage
```

5. Alsu driver:

```
package alsu_driver_pkg;

  import uvm_pkg::*;
  `include "uvm_macros.svh"

  import alsu_seq_item_pkg::*;
  import alsu_config_pkg::*;

  class alsu_driver extends uvm_driver #(alsu_seq_item);
    `uvm_component_utils(alsu_driver)

    virtual alsu_if alsu_vif;
    alsu_seq_item stim_seq_item;

    function new(string name = "alsu_driver", uvm_component parent = null);
      super.new(name, parent);
    endfunction

    task run_phase(uvm_phase phase);
      super.run_phase(phase);
      forever begin
        stim_seq_item = alsu_seq_item::type_id::create("stim_seq_item");
        seq_item_port.get_next_item(stim_seq_item);
        alsu_vif.rst=stim_seq_item.rst;
            alsu_vif.serial_in=stim_seq_item.serial_in;
            alsu_vif.direction=stim_seq_item.direction;
            alsu_vif.cin=stim_seq_item.cin;
           alsu_vif.opcode= stim_seq_item.op_code;
           alsu_vif.red_op_A= stim_seq_item.red_op_a;
            alsu_vif.red_op_B=stim_seq_item.red_op_b;
           alsu_vif.bypass_A= stim_seq_item.bypass_a;
           alsu_vif.bypass_B= stim_seq_item.bypass_b;
           alsu_vif.A= stim_seq_item.A;
           alsu_vif.B= stim_seq_item.B;
         repeat (2) @(negedge alsu_vif.clk);
         seq_item_port.item_done();
        `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
      end
    endtask

  endclass
endpackage
```

6. Alsu env:

```systemverilog
package alsu_env_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import alsu_agent_pkg::*;
    import alsu_scoreboard_pkg::*;
    import alsu_coverage_pkg::*;
    class alsu_env extends uvm_env;
        `uvm_component_utils(alsu_env)

        alsu_agent agent;
        alsu_scoreboard score_b;
        alsu_coverage coverage;

        function new(string name = "shift_reg_env", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            agent = alsu_agent::type_id::create("agent",this);
            score_b = alsu_scoreboard::type_id::create("score_b",this);
            coverage = alsu_coverage::type_id::create("coverage",this);
        endfunction: build_phase

        function void connect_phase(uvm_phase phase);
            agent.agent_ap.connect(score_b.sb_export);
            agent.agent_ap.connect(coverage.coverage_export);
        endfunction
    endclass
endpackage
```

7. alsu_if :

```systemverilog
import shared_pkg::*;

interface alsu_if (clk);
  input clk;
logic  cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
OP_CODE opcode;
logic signed [2:0] A, B;
logic [15:0] leds;
logic signed [5:0] out;
logic signed [5:0] expected_out;
logic [15:0] leds_expected;
modport DUT (input clk,cin,rst,red_op_A,red_op_B,bypass_A,bypass_B,direction,serial_in,opcode,A,B,output leds,out);
    modport GOLDEN (
          input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B,
          direction, serial_in, opcode, A, B,
          output expected_out, leds_expected
      );
endinterface
```

8. Alsu monitor:

```
package alsu_monitor_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import shared_pkg::*;
    import alsu_seq_item_pkg::*;
    class alsu_monitor extends uvm_monitor;
        `uvm_component_utils(alsu_monitor)
        virtual alsu_if alsu_vif;
        alsu_seq_item rsp_seq_item;
        uvm_analysis_port #(alsu_seq_item) monitor_ap;

        function new(string name = "shift_reg_monitor", uvm_component parent = null) ;
            super.new(name, parent);
        endfunction
        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            monitor_ap = new("monitor_ap", this);
        endfunction: build_phase
        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                rsp_seq_item = alsu_seq_item::type_id::create("rsp_seq_item");
                 repeat(2) @(negedge alsu_vif.clk);
                 rsp_seq_item.rst=alsu_vif.rst;
                rsp_seq_item.serial_in=alsu_vif.serial_in;
                rsp_seq_item.direction=alsu_vif.direction;
                rsp_seq_item.cin=alsu_vif.cin;
                rsp_seq_item.op_code=alsu_vif.opcode;
                rsp_seq_item.red_op_a=alsu_vif.red_op_A;
                rsp_seq_item.red_op_b=alsu_vif.red_op_B;
                rsp_seq_item.bypass_a=alsu_vif.bypass_A;
                rsp_seq_item.bypass_b=alsu_vif.bypass_B;
                rsp_seq_item.A=alsu_vif.A;
                rsp_seq_item.B=alsu_vif.B;
                rsp_seq_item.out=alsu_vif.out;
                rsp_seq_item.leds=alsu_vif.leds;
                rsp_seq_item.expected_out=alsu_vif.expected_out;
                rsp_seq_item.leds_expected=alsu_vif.leds_expected;
                monitor_ap.write(rsp_seq_item);
                `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
            end
        endtask
    endclass
endpackage
```

9. Alsu main seq :

```
package alsu_main_seq_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import alsu_seq_item_pkg::*;
    import shared_pkg::*;
    class alsu_main_sequence extends uvm_sequence #(alsu_seq_item);
        `uvm_object_utils(alsu_main_sequence);
        alsu_seq_item seq_item;
        function new (string name = "alsu_main_sequence");
            super.new(name);
        endfunction
        task body;
            repeat(20000) begin
                seq_item = alsu_seq_item::type_id::create("seq_item");
                 ////disable constraints number 8//////
                 seq_item.X.constraint_mode(0);
                start_item(seq_item);
                assert(seq_item.randomize());
                finish_item(seq_item);
            end
            repeat(1000) begin
                  seq_item = alsu_seq_item::type_id::create("seq_item");
                   ////disable all constraints //////
                     seq_item.constraint_mode(0);
                       ////enable constraint X //////
                 seq_item.X.constraint_mode(1);

                assert(seq_item.randomize());
                     ///directed value for some inputs///////
             seq_item.rst=0;
             seq_item.bypass_a=0;
            seq_item.bypass_b=0;
            seq_item.red_op_a=0;
            seq_item.red_op_b=0;
                  for (int i=0;i<6;i=i+1) begin
                    start_item(seq_item);
                    seq_item.op_code=seq_item.array[i];
                     finish_item(seq_item);
                end
            end
        endtask
    endclass
endpackage
```

10. Alsu reset seq:

```
package alsu_reset_seq_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"
import shared_pkg::*;
    import alsu_seq_item_pkg::*;
    class alsu_reset_sequence extends uvm_sequence #(alsu_seq_item);
        `uvm_object_utils(alsu_reset_sequence);
        alsu_seq_item seq_item;

        function new (string name = "alsu_reset_sequence");
            super.new(name);
        endfunction

        task body;
            seq_item = alsu_seq_item::type_id::create("seq_item");
            start_item(seq_item);
            seq_item.rst = 1;
           seq_item.serial_in=0;
           seq_item.A=0;
           seq_item.B=0;
           seq_item.cin=0;
           seq_item.direction=0;
           seq_item.op_code=OR;
           seq_item.red_op_a=0;
           seq_item.red_op_b=0;
           seq_item.bypass_a=0;
           seq_item.bypass_b=0;
            finish_item(seq_item);
        endtask
    endclass
endpackage
```

11. Alsu scoreboard:

```
package alsu_scoreboard_pkg;
import uvm_pkg::*;
  `include "uvm_macros.svh"
  import alsu_seq_item_pkg::*;
  import shared_pkg::*;
    /*....................expected out declaration................*/
      logic signed [5:0] expected_out;
      logic [15:0] leds_expected;
  class alsu_scoreboard extends uvm_scoreboard;
      `uvm_component_utils(alsu_scoreboard)
      uvm_analysis_export #(alsu_seq_item) sb_export;
      uvm_tlm_analysis_fifo #(alsu_seq_item) sb_fifo;
      alsu_seq_item seq_item_sb;
     int error_count = 0;
    int correct_count = 0;
      /*......................................................*/
      function new(string name = "alsu_scoreboard", uvm_component parent = null) ;
        super.new(name, parent);
      endfunction

      function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        sb_export = new("sb_export", this);
        sb_fifo = new("sb_fifo", this);
      endfunction

      function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        sb_export.connect(sb_fifo.analysis_export);
      endfunction

      task run_phase(uvm_phase phase);
        super.run_phase(phase);
        forever begin
            sb_fifo.get(seq_item_sb);
            if ((seq_item_sb.out == seq_item_sb.expected_out)&&(seq_item_sb.leds==seq_item_sb.leds_expected)) begin
               `uvm_info("run_phase", $sformatf("%s", seq_item_sb.convert2string()), UVM_LOW);
`uvm_info("run_phase", $sformatf("expected_out value : %b , leds_expected : %b", seq_item_sb.expected_out, seq_item_sb.leds_expected), UVM_
              correct_count++;
            end
            else begin
            `uvm_error("run_phase", $sformatf("Comparsion failed, Transaction received by the DUT:%s  While the expected out:0b%0b & leds_e
            error_count++;
            end

          end
    endtask
  function void report_phase(uvm_phase phase);
        super.report_phase(phase);
        `uvm_info("report_phase", $sformatf("Total successful transactions: %0d",correct_count), UVM_LOW);
        `uvm_info("report_phase", $sformatf("Total failed transactions: %0d",error_count), UVM_LOW);
      endfunction
  endclass
endpackage
```

12. Alsu seq item :

```
package alsu_seq_item_pkg;

    import uvm_pkg::*;
    import shared_pkg::*;
    `include "uvm_macros.svh"

    /*................class...................*/
    class alsu_seq_item extends uvm_sequence_item;
        `uvm_object_utils(alsu_seq_item)

/*....................define variables............................*/
        rand logic rst;
        rand logic  signed [2:0] A,B;
        rand OP_CODE op_code;
        rand logic cin,serial_in,direction;
        rand logic red_op_a,red_op_b;
        rand logic bypass_a,bypass_b;
         rand OP_CODE array [6];
         logic signed [5:0] out;
         logic [15:0] leds;
         logic signed [5:0] expected_out;
         logic [15:0] leds_expected;

         /*..........................*/

        function new(string name = "alsu_seq_item");
            super.new(name);
        endfunction

        function string convert2string();
            return $sformatf("%s rst = %b, A = %d, B = %d,op_code= %s, cin = %b,serial_in=%b,direction=%b,red_op_a=%b,red_op_b=%b,bypass_a
        endfunction

        function string convert2string_stimulus();
            return $sformatf(" rst = %b, A = %d, B = %d,op_code= %s, cin = %b,serial_in=%b,direction=%b,red_op_a=%b,red_op_b=%b,bypass_a=%
        endfunction
/*.....................................................constraints...........................................................*/
/*........constraint to force rst to be low most of the time...*/
constraint RST {
    rst dist{0:=99,1:=1};
}
/*........op code constraint........*/
constraint OP {
    op_code dist{OR:=30,XOR:=10,[ADD:ROTATE]:/45,[INVALID_6:INVALID_7]:/5};
}
/*...........constraint A,B.......*/
constraint A_B {
/*................forc A,B to be MAX,MIN,ZERO 90% of the time.....*/
if (op_code==ADD||op_code==MULT)
    {
    A dist{ MAXNEG:=20,MAXPOS:=40,0:=30, [-3:-1]:/5,[1:2]:/5};
    B dist{ MAXNEG:=20,MAXPOS:=40,0:=30, [-3:-1]:/5,[1:2]:/5};
}
/*.........constraints on A when OR,XOR opcode and red_op_a=1.....*/
```

```
}
/*.........constraints on A when OR,XOR opcode and red_op_a=1.....*/
if((op_code==OR||op_code==XOR)&red_op_a)
    {
A dist {3'b001:=20,3'b010:=25,3'b100:=40,0:=5,3:=5,[-4:-1]:/5};
B==0;
}
/*.........constraints on B when OR,XOR opcode and red_op_b=1.....*/
if((op_code==OR||op_code==XOR)&red_op_b)
    {
B dist {  3'b001:=20,3'b010:=25,3'b100:=40, 0:=5,3:=5,[-4:-1]:/5};
A==0;
}
}

/*..............red_op_A,red_op_B constraints......*/
constraint red_op_A {
red_op_a dist {0:=5,1:=95};
}
constraint red_op_B {
red_op_b dist {0:=5,1:=95};
}
/*.................bypass A,bypass B must be disabled most of the time.......*/
constraint bypass_A {
bypass_a dist {0:=95,1:=5};
}
constraint bypass_B {
bypass_b dist {0:=95,1:=5};
}
///////////////////////unique constraint for the array///////////////////////
constraint X {
    foreach (array[i]){
        array[i]!={INVALID_6,INVALID_7};
        if(i==0)
        array[i]==OR;
           else if (i > 0) {
            (array[i-1] == OR) -> (array[i] == XOR);
            (array[i-1] == XOR) -> (array[i] == ADD);
            (array[i-1] == ADD) -> (array[i] == MULT);
            (array[i-1] == MULT) -> (array[i] == SHIFT);
            (array[i-1] == SHIFT) -> (array[i] == ROTATE);
        }

        }
}

    endclass


endpackage
```

13. Alsu seqr:

```
package alsu_sequencer_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    import alsu_seq_item_pkg::*;
    class alsu_sequencer extends uvm_sequencer #(alsu_seq_item);
        `uvm_component_utils(alsu_sequencer);

        function new (string name = "alsu_sequencer", uvm_component parent = null);
            super.new(name, parent);
        endfunction
    endclass

endpackage
```

14. Alsu sva :

```
mport shared_pkg::*;
odule alsu_sva (alsu_if.DUT a_if);
*.......................................OUT assertion............................*/
 a1 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (a_if.bypass_A & a_if.bypass_B&INPUT_PRIORITY == "A") |-> ##2 (a_if.out == $past(a_if.A,2)));
c1 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (a_if.bypass_A & a_if.bypass_B
INPUT_PRIORITY == "A") |-> ##2 (a_if.out == $past(a_if.A,2)) );

  a2 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (a_if.bypass_A & !a_if.bypass_B
 ) |-> ##2 (a_if.out == $past(a_if.A,2)) );
c2 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (a_if.bypass_A
  & !a_if.bypass_B) |-> ##2 (a_if.out == $past(a_if.A,2)) );

  a3 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & a_if.bypass_B
 |-> ##2 (a_if.out == $past(a_if.B,2)));
c3 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & a_if.bypass_B
 |-> ##2 (a_if.out == $past(a_if.B,2)) );

 a4 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  & (((a_if.red_op_A || a_if.red_op_B) & (a_if.opcode != OR & a_if.opcode != XOR))
     || (a_if.opcode == INVALID_6 & a_if.opcode == INVALID_7))) |-> ##2 (a_if.out == 0) );
c4 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  & (((a_if.red_op_A || a_if.red_op_B) & (a_if.opcode != OR & a_if.opcode != XOR))
     || (a_if.opcode == INVALID_6 & a_if.opcode == INVALID_7))) |-> ##2 (a_if.out == 0) );

   a5 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A
     & !a_if.bypass_B & a_if.opcode == OR & a_if.red_op_A & a_if.red_op_B&INPUT_PRIORITY == "A") |-> ##2 (a_if.out == |$past(a_if.A,2))&(a_if.leds==0) );
c5 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A
  & !a_if.bypass_B & a_if.opcode == OR & a_if.red_op_A & a_if.red_op_B&INPUT_PRIORITY == "A") |-> ##2 (a_if.out == |$past(a_if.A,2))&(a_if.leds==0) );

   a6 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A
     & !a_if.bypass_B & a_if.opcode == OR & !a_if.red_op_A & a_if.red_op_B) |-> ##2 (a_if.out == |$past(a_if.B,2))&(a_if.leds==0) );
c6 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A
 & !a_if.bypass_B & a_if.opcode == OR & !a_if.red_op_A & a_if.red_op_B) |-> ##2 (a_if.out == |$past(a_if.B,2))&(a_if.leds==0) );

   a7 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
     & a_if.opcode == OR & a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == |$past(a_if.A,2))&(a_if.leds==0) );
c7 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  & a_if.opcode == OR & a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == |$past(a_if.A,2))&(a_if.leds==0) );

   a8 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
    & a_if.opcode == OR & !a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == ($past(a_if.A,2) | $past(a_if.B,2)))&(a_if.leds==0) );
c8 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  & a_if.opcode == OR & !a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == ($past(a_if.B,2) | $past(a_if.A,2)))&(a_if.leds==0) );

  a9 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
     & a_if.opcode == XOR & a_if.red_op_A & a_if.red_op_B&INPUT_PRIORITY == "A") |-> ##2 (a_if.out == ^$past(a_if.A,2)) &(a_if.leds==0));
c9 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  & a_if.opcode == XOR & a_if.red_op_A & a_if.red_op_B&INPUT_PRIORITY == "A") |-> ##2 (a_if.out == ^$past(a_if.A,2))&(a_if.leds==0) );

   a10 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
     & a_if.opcode == XOR & !a_if.red_op_A & a_if.red_op_B) |-> ##2 (a_if.out == ^$past(a_if.B,2))&(a_if.leds==0) );
c10 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  & a_if.opcode == XOR & !a_if.red_op_A & a_if.red_op_B) |-> ##2 (a_if.out == ^$past(a_if.B,2))&(a_if.leds==0) );

   a11 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
    & a_if.opcode == XOR & a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == ^$past(a_if.A,2))&(a_if.leds==0) );
c11 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  & a_if.opcode == XOR & a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == ^$past(a_if.A,2)) &(a_if.leds==0));

   a12 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
     & a_if.opcode == XOR & !a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == ($past(a_if.B,2) ^ $past(a_if.A,2)))&(a_if.leds==0) );
c12 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  & a_if.opcode == XOR & !a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == ($past(a_if.B,2) ^ $past(a_if.A,2)))&(a_if.leds==0));

   a13 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
     &a_if.opcode == ADD& !a_if.red_op_A & !a_if.red_op_B&FULL_ADDER=="ON") |-> ##2 (a_if.out == ($past(a_if.B,2) + $past(a_if.A,2) + $past(a_if.cin,2))) &(a_if.leds==0));
c13 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  &a_if.opcode == ADD& !a_if.red_op_A & !a_if.red_op_B&FULL_ADDER=="ON") |-> ##2 (a_if.out == ($past(a_if.A,2) + $past(a_if.B,2)+ $past(a_if.cin,2))) &(a_if.leds==0));

    a14 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
     &a_if.opcode == MULT& !a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == ($past(a_if.B,2) * $past(a_if.A,2)))&(a_if.leds==0) );
c14 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
  &a_if.opcode == MULT& !a_if.red_op_A & !a_if.red_op_B) |-> ##2 (a_if.out == ($past(a_if.B,2) * $past(a_if.A,2)))&(a_if.leds==0) );

     a15 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
       &!a_if.red_op_A & !a_if.red_op_B&a_if.opcode==SHIFT&a_if.direction==1)|-> ##2 (a_if.out=={$past(a_if.out[4:0],1),$past(a_if.serial_in,2)})&(a_if.leds==0) );
     c15 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
       &!a_if.red_op_A & !a_if.red_op_B&a_if.opcode==SHIFT&a_if.direction==1)|-> ##2 (a_if.out=={$past(a_if.out[4:0],1),$past(a_if.serial_in,2)})&(a_if.leds==0) );


      a16 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
       &!a_if.red_op_A & !a_if.red_op_B&a_if.opcode==SHIFT&a_if.direction==0)|-> ##2 (a_if.out=={$past(a_if.serial_in,2),$past(a_if.out[5:1],1)})&(a_if.leds==0) );
     c16 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
       &!a_if.red_op_A & !a_if.red_op_B&a_if.opcode==SHIFT&a_if.direction==0)|-> ##2 (a_if.out=={$past(a_if.serial_in,2),$past(a_if.out[5:1],1)}) &(a_if.leds==0));

           a17 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
             &!a_if.red_op_A & !a_if.red_op_B&a_if.opcode==ROTATE&a_if.direction==1)|-> ##2 (a_if.out == {$past(a_if.out[4:0],1), $past(a_if.out[5],1)})&(a_if.leds==0) );
     c17 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
       &!a_if.red_op_A & !a_if.red_op_B&a_if.opcode==ROTATE&a_if.direction==1)|-> ##2 (a_if.out=={$past(a_if.out[4:0],1), $past(a_if.out[5],1)}) &(a_if.leds==0));

      a18 : assert property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B&!a_if.red_op_A
       & !a_if.red_op_B&a_if.opcode==ROTATE&a_if.direction==0)|-> ##2 (  a_if.out == {$past(a_if.out[0],1), $past(a_if.out[5:1],1)}) &(a_if.leds==0));
     c18 : cover property   (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
       &!a_if.red_op_A & !a_if.red_op_B&a_if.opcode==ROTATE&a_if.direction==0)|-> ##2 (  a_if.out == {$past(a_if.out[0],1), $past(a_if.out[5:1],1)})&(a_if.leds==0) );

19: assert property (@(posedge a_if.clk)  disable iff(a_if.rst) ( (((a_if.red_op_A || a_if.red_op_B) & (a_if.opcode != OR & a_if.opcode != XOR))
     || (a_if.opcode == INVALID_6 & a_if.opcode == INVALID_7))) |-> ##2 (a_if.leds==~$past(a_if.leds,1)));

19:cover property (@(posedge a_if.clk) disable iff(a_if.rst) ( (((a_if.red_op_A || a_if.red_op_B) & (a_if.opcode != OR & a_if.opcode != XOR))
     || (a_if.opcode == INVALID_6 & a_if.opcode == INVALID_7))) |-> ##2 (a_if.leds==~$past(a_if.leds,1)));
*.......................................reset assertion.....................*/
*...............reset assertion.......*/
lways_comb begin
 if(a_if.rst)
 _1:assert final (a_if.out==0&&a_if.leds==0);
nd
ndmodule
```

15. Golden model:

```
import shared_pkg::*;
module golden_design (alsu_if.GOLDEN a_if);
  reg red_op_A_reg_ex, red_op_B_reg_ex, bypass_A_reg_ex, bypass_B_reg_ex, direction_reg_ex, serial_in_reg_ex;
reg  [1:0] cin_reg_ex;
reg [2:0] opcode_reg_ex;
reg signed [2:0] A_reg_ex, B_reg_ex;
wire invalid_red_op_ex, invalid_opcode_ex, invalid_ex;
//Invalid handling
assign invalid_red_op_ex = (red_op_A_reg_ex || red_op_B_reg_ex) & (opcode_reg_ex[1] || opcode_reg_ex[2]);
assign invalid_opcode_ex = opcode_reg_ex[1] & opcode_reg_ex[2];
assign invalid = invalid_red_op_ex || invalid_opcode_ex;
//Registering input signals
always @(posedge a_if.clk or posedge a_if.rst) begin
  if(a_if.rst) begin
     cin_reg_ex <= 0;
     red_op_B_reg_ex <= 0;
     red_op_A_reg_ex <= 0;
     bypass_B_reg_ex <= 0;
     bypass_A_reg_ex <= 0;
     direction_reg_ex <= 0;
     serial_in_reg_ex <= 0;
     opcode_reg_ex <= 0;
     A_reg_ex <= 0;
     B_reg_ex <= 0;
  end else begin
     cin_reg_ex <= a_if.cin;
     red_op_B_reg_ex <= a_if.red_op_B;
     red_op_A_reg_ex <= a_if.red_op_A;
     bypass_B_reg_ex <= a_if.bypass_B;
     bypass_A_reg_ex <= a_if.bypass_A;
     direction_reg_ex <= a_if.direction;
     serial_in_reg_ex <= a_if.serial_in;
     opcode_reg_ex <= a_if.opcode;
     A_reg_ex <= a_if.A;
     B_reg_ex <= a_if.B;
  end
end

//leds output blinking
always @(posedge a_if.clk or posedge a_if.rst) begin
  if(a_if.rst) begin
     a_if.leds_expected <= 0;
  end else begin
      if (invalid)
        a_if.leds_expected <= ~a_if.leds_expected;
      else
        a_if.leds_expected <= 0;
  end
end

//ALSU output processing
always @(posedge a_if.clk or posedge a_if.rst) begin
  if(a_if.rst) begin
    a_if.expected_out <= 0;
  end
  else begin
    if (bypass_A_reg_ex && bypass_B_reg_ex)
      a_if.expected_out <= (INPUT_PRIORITY == "A")? A_reg_ex: B_reg_ex;
    else if (bypass_A_reg_ex)
      a_if.expected_out<= A_reg_ex;
    else if (bypass_B_reg_ex)
      a_if.expected_out <= B_reg_ex;
    else if (invalid)
        a_if.expected_out <= 0;
    else begin
        case ( opcode_reg_ex)
          3'h0: begin
            if (red_op_A_reg_ex && red_op_B_reg_ex)
              a_if.expected_out <= (INPUT_PRIORITY == "A")? |A_reg_ex: |B_reg_ex;
            else if (red_op_A_reg_ex)
              a_if.expected_out <= |A_reg_ex;
            else if (red_op_B_reg_ex)
              a_if.expected_out <= |B_reg_ex;
            else
              a_if.expected_out <= A_reg_ex | B_reg_ex;
          end
          3'h1: begin
            if (red_op_A_reg_ex && red_op_B_reg_ex)
              a_if.expected_out <= (INPUT_PRIORITY == "A")? ^A_reg_ex: ^B_reg_ex;
            else if (red_op_A_reg_ex)
              a_if.expected_out <= ^A_reg_ex;
            else if (red_op_B_reg_ex)
              a_if.expected_out <= ^B_reg_ex;
            else
              a_if.expected_out <= A_reg_ex ^ B_reg_ex;
          end
          3'h2: begin
            if(FULL_ADDER=="ON")
          a_if.expected_out <= A_reg_ex + B_reg_ex+cin_reg_ex;
        else
          a_if.expected_out <= A_reg_ex + B_reg_ex;
      end
          3'h3: a_if.expected_out <= A_reg_ex * B_reg_ex;
          3'h4: begin
            if (direction_reg_ex)
              a_if.expected_out <= {a_if.expected_out[4:0], serial_in_reg_ex};
            else
              a_if.expected_out <= {serial_in_reg_ex, a_if.expected_out[5:1]};
          end
          3'h5: begin
            if (direction_reg_ex)
              a_if.expected_out <= {a_if.expected_out[4:0], a_if.expected_out[5]};
            else
              a_if.expected_out <= {a_if.expected_out[0], a_if.expected_out[5:1]};
          end
        endcase
    end
  end
end
endmodule
```

16. test:

```
package alsu_test_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"
    import alsu_config_pkg::*;
    import shift_reg_env_pkg::*;
    import shift_reg_config_pkg::*;
    import alsu_main_seq_pkg::*;
    import alsu_reset_seq_pkg::*;
        import alsu_agent_pkg::*;
        import shift_reg_main_seq_pkg::*;
         import shift_reg_agent_pkg::*;
    import alsu_env_pkg::*;
    class alsu_test extends uvm_test;
        `uvm_component_utils(alsu_test)
        alsu_agent agent;
        alsu_config alsu_cfg;
        shift_reg_config shift_cfg;
        virtual alsu_if alsu_vif;
        virtual shift_reg_if shift_reg_vif;
        alsu_main_sequence main_seq;
        alsu_reset_sequence reset_seq;
        alsu_env a_env;
        shift_reg_env s_env;

        function new(string name = "alsu_test", uvm_component parent = null);
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            agent = alsu_agent::type_id::create("agent",this);
            a_env = alsu_env::type_id::create("a_env",this);
            s_env = shift_reg_env::type_id::create("s_env",this);
            alsu_cfg = alsu_config::type_id::create("alsu_cfg");
            shift_cfg = shift_reg_config::type_id::create("shift_cfg");
            main_seq = alsu_main_sequence::type_id::create("main_seq",this);
            reset_seq = alsu_reset_sequence::type_id::create("reset_seq",this);
            /*.....................................alsu db....................................*/
            if (!uvm_config_db #(virtual alsu_if)::get(this, "","ALSU_IF", alsu_cfg.alsu_vif))
                `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the alsu from the uvm_config_db");

  /*...................................shift reg db..............................*/
   if (!uvm_config_db #(virtual shift_reg_if)::get(this, "","SHIFT_IF", shift_cfg.shift_reg_vif))
                `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the shift from the uvm_config_db");
            uvm_config_db #(shift_reg_config)::set(this, "*", "CFG", shift_cfg);
            uvm_config_db #(alsu_config)::set(this, "*", "CFG", alsu_cfg);
            //////////////////////////////////////////////////////////////
             shift_cfg.is_active=UVM_PASSIVE;
            alsu_cfg.is_active=UVM_ACTIVE;
        endfunction
        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            phase.raise_objection(this);
            //reset sequence
            `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
```


```
        endfunction
        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            phase.raise_objection(this);
            //reset sequence
            `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
```


```
            reset_seq.start(agent.sqr);
            `uvm_info("run_phase", "Reset Deasserted", UVM_LOW)
            //main sequence
            `uvm_info("run_phase", "Stimulus Generation Started", UVM_LOW)
            main_seq.start(agent.sqr);
            `uvm_info("run_phase", "Stimulus Generation Ended", UVM_LOW)
            phase.drop_objection(this);
        endtask
    endclass
endpackage
```

17. top:

```
import uvm_pkg::*;
`include "uvm_macros.svh"
import alsu_test_pkg::*;
module alsu_top();
  // Clock generation
  bit clk;
  initial begin
    forever #1 clk=~clk;
  end
  // Instantiate the interface and DUT
 alsu_if a_if (clk);
  ALSU dut ( a_if);
  golden_design ex ( a_if);
  bind ALSU alsu_sva sva_inst (a_if);
  shift_reg_if shiftregif();
  shift_reg DUT(shiftregif);
  assign shiftregif.datain=a_if.out;
  assign dut.shift_out_reg=shiftregif.dataout;
  assign shiftregif.serial_in=dut.serial_in_reg;
  assign shiftregif.direction=dut.direction_reg;
  assign shiftregif.mode=dut.opcode_reg[0];
initial begin
//......configuration database................//
 uvm_config_db #(virtual alsu_if) ::set (null,"uvm_test_top","ALSU_IF",a_if);
  uvm_config_db #(virtual shift_reg_if)::set(null, "uvm_test_top", "SHIFT_IF", shiftregif);
//............uvm run task...............//
  run_test("alsu_test");
end
endmodule
```

18. Shared pkg:

```
package shared_pkg;
      /*......................parameters...................*/
 parameter INPUT_PRIORITY="A";
    parameter FULL_ADDER="ON";
    parameter MAXPOS=3;
    parameter MAXNEG=-4;
    typedef enum  {OR,XOR,ADD,MULT,SHIFT,ROTATE,INVALID_6,INVALID_7} OP_CODE;
    typedef enum {RIGHT, LEFT} direction_e;
endpackage
```

19. Shift reg design:

```
//
/////////////////////////////////////////////////////////////////////////////////
module shift_reg (shift_reg_if.DUT shiftregif);


always @(*) begin

      if (shiftregif.mode) // rotate
         if (shiftregif.direction) // left
            shiftregif.dataout = {shiftregif.datain[4:0], shiftregif.datain[5]};
         else
            shiftregif.dataout = {shiftregif.datain[0], shiftregif.datain[5:1]};
      else // shift
         if (shiftregif.direction) // left
            shiftregif.dataout = {shiftregif.datain[4:0], shiftregif.serial_in};
         else
            shiftregif.dataout = {shiftregif.serial_in, shiftregif.datain[5:1]};
end
endmodule
```

20. Shft reg agent :

```
package shift_reg_agent_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

    import shift_reg_sequencer_pkg::*;
    import shift_reg_monitor_pkg::*;
    import shift_reg_seq_item_pkg::*;
    import shift_reg_driver_pkg::*;
    import shift_reg_config_pkg::*;

    class shift_reg_agent extends uvm_agent;
        `uvm_component_utils(shift_reg_agent)
        shift_reg_sequencer sqr_s;
        shift_reg_driver drv;
        shift_reg_monitor mon;
        shift_reg_config shift_reg_cfg;
        uvm_analysis_port #(shift_reg_seq_item) agt_ap;

        function new (string name = "shift_reg_agent", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            shift_reg_cfg = shift_reg_config::type_id::create("shift_reg_cfg");
            agt_ap = new("agt_ap", this);
            if (!uvm_config_db #(shift_reg_config)::get(this , "", "CFG", shift_reg_cfg))begin
                `uvm_fatal("build_phase" , "Unable to get configuration object")
            end
          if(shift_reg_cfg.is_active==UVM_ACTIVE) begin
            sqr_s = shift_reg_sequencer::type_id::create("sqr_s",this);
            drv = shift_reg_driver::type_id::create("drv",this);
            mon = shift_reg_monitor::type_id::create("mon",this);
          end
            else begin
            mon = shift_reg_monitor::type_id::create("mon",this);
            end
        endfunction

        function void connect_phase(uvm_phase phase);
            if(shift_reg_cfg.is_active==UVM_ACTIVE) begin
            drv.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
            drv.seq_item_port.connect(sqr_s.seq_item_export);
            mon.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
            mon.mon_ap.connect(agt_ap);
        end
else begin
            mon.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
            mon.mon_ap.connect(agt_ap);
        end
        endfunction
    endclass

endpackage
```

21. Shift reg config:

```
package shift_reg_config_pkg;
    import uvm_pkg::*;
    `include "uvm_macros.svh"

    class shift_reg_config extends uvm_object;
        `uvm_object_utils(shift_reg_config)

        virtual shift_reg_if shift_reg_vif;
  uvm_active_passive_enum is_active;
        function new(string name = "shift_reg_config");
            super.new(name);
        endfunction
    endclass
endpackage : shift_reg_config_pkg
```

22. Shift reg coverage :

```
package shift_reg_coverage_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    import shift_reg_seq_item_pkg::*;

    class shift_reg_coverage extends uvm_component;
        `uvm_component_utils(shift_reg_coverage)
        uvm_analysis_export #(shift_reg_seq_item) cov_export;
        uvm_tlm_analysis_fifo #(shift_reg_seq_item) cov_fifo;
        shift_reg_seq_item seq_item_cov;

        covergroup cvr_grp;
            datain_cvp : coverpoint seq_item_cov.datain;
            serialin_cvp : coverpoint seq_item_cov.serial_in;
            direction_cvp : coverpoint seq_item_cov.direction;
            mode_cvp : coverpoint seq_item_cov.mode;
        endgroup

        function new(string name = "shift_reg_coverage", uvm_component parent = null) ;
            super.new(name, parent);
            cvr_grp = new();
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            cov_export = new("cov_export", this);
            cov_fifo = new("cov_fifo", this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            cov_export.connect(cov_fifo.analysis_export);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                cov_fifo.get(seq_item_cov);
                cvr_grp.sample();
            end
        endtask

    endclass
endpackage
```

23. Shift reg driver:

```
package shift_reg_driver_pkg;

  import uvm_pkg::*;
  `include "uvm_macros.svh"

  import shift_reg_seq_item_pkg::*;
  import shift_reg_config_pkg::*;

  class shift_reg_driver extends uvm_driver #(shift_reg_seq_item);
    `uvm_component_utils(shift_reg_driver)

    virtual shift_reg_if shift_reg_vif;
    shift_reg_seq_item stim_seq_item;

    function new(string name = "shift_reg_driver", uvm_component parent = null);
      super.new(name, parent);
    endfunction

    task run_phase(uvm_phase phase);
      super.run_phase(phase);
      forever begin
        stim_seq_item = shift_reg_seq_item::type_id::create("stim_seq_item");
        seq_item_port.get_next_item(stim_seq_item);
        shift_reg_vif.direction = stim_seq_item.direction;
        shift_reg_vif.mode = stim_seq_item.mode;
        shift_reg_vif.datain = stim_seq_item.datain;
        shift_reg_vif.serial_in = stim_seq_item.serial_in;
       #2;
        seq_item_port.item_done();
        `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
      end
    endtask

  endclass
endpackage
```

24. Shift reg env:

```
package shift_reg_env_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    import shift_reg_agent_pkg::*;
    import shift_reg_scoreboard_pkg::*;
    import shift_reg_coverage_pkg::*;
    class shift_reg_env extends uvm_env;
        `uvm_component_utils(shift_reg_env)

        shift_reg_agent agt;
        shift_reg_scoreboard sb;
        shift_reg_coverage cov;

        function new(string name = "shift_reg_env", uvm_component parent = null);
            super.new(name,parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            agt = shift_reg_agent::type_id::create("agt",this);
            sb = shift_reg_scoreboard::type_id::create("sb",this);
            cov = shift_reg_coverage::type_id::create("cov",this);
        endfunction: build_phase

        function void connect_phase(uvm_phase phase);
            agt.agt_ap.connect(sb.sb_export);
            agt.agt_ap.connect(cov.cov_export);
        endfunction
    endclass
endpackage
```

25. Shift reg if :

```
interface shift_reg_if ();

    bit  serial_in, direction, mode;
    logic [5:0] datain;
    logic [5:0] dataout;

    modport DUT (input   serial_in, direction, mode, datain, output dataout);


endinterface
```

26. Shift reg main seq:

```
package shift_reg_main_seq_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    import shift_reg_seq_item_pkg::*;
    import shared_pkg::*;
    class shift_reg_main_sequence extends uvm_sequence #(shift_reg_seq_item);
        `uvm_object_utils(shift_reg_main_sequence);
        shift_reg_seq_item seq_item;

        function new (string name = "shift_reg_main_sequence");
            super.new(name);
        endfunction

        task body;
            repeat(26000) begin
                seq_item = shift_reg_seq_item::type_id::create("seq_item");
                start_item(seq_item);
                assert(seq_item.randomize());
                finish_item(seq_item);
            end
        endtask
    endclass
endpackage
```

27. Shift reg monitor :

```
package shift_reg_monitor_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    import shared_pkg::*;

    import shift_reg_seq_item_pkg::*;
    class shift_reg_monitor extends uvm_monitor;
        `uvm_component_utils(shift_reg_monitor)
        virtual shift_reg_if shift_reg_vif;
        shift_reg_seq_item rsp_seq_item;
        uvm_analysis_port #(shift_reg_seq_item) mon_ap;

        function new(string name = "shift_reg_monitor", uvm_component parent = null) ;
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            mon_ap = new("mon_ap", this);

        endfunction: build_phase

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                rsp_seq_item = shift_reg_seq_item::type_id::create("rsp_seq_item");
                #2;
                rsp_seq_item.direction = direction_e'(shift_reg_vif.direction);
                rsp_seq_item.mode = (shift_reg_vif.mode);
                rsp_seq_item.datain = shift_reg_vif.datain;
                rsp_seq_item.serial_in = shift_reg_vif.serial_in;
                rsp_seq_item.dataout = shift_reg_vif.dataout;
                mon_ap.write(rsp_seq_item);
                `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
            end
        endtask
    endclass
endpackage : shift_reg_monitor_pkg
```

28. Shift reg scoreboard:

```
package shift_reg_scoreboard_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    import shift_reg_seq_item_pkg::*;
    import shared_pkg::*;

    class shift_reg_scoreboard extends uvm_scoreboard;
        `uvm_component_utils(shift_reg_scoreboard)
        uvm_analysis_export #(shift_reg_seq_item) sb_export;
        uvm_tlm_analysis_fifo #(shift_reg_seq_item) sb_fifo;
        shift_reg_seq_item seq_item_sb;
        logic [5:0] shift_reg_out_ref;

        int error_count = 0;
        int correct_count = 0;

        function new(string name = "shift_reg_scoreboard", uvm_component parent = null) ;
            super.new(name, parent);
        endfunction

        function void build_phase(uvm_phase phase);
            super.build_phase(phase);
            sb_export = new("sb_export", this);
            sb_fifo = new("sb_fifo", this);
        endfunction

        function void connect_phase(uvm_phase phase);
            super.connect_phase(phase);
            sb_export.connect(sb_fifo.analysis_export);
        endfunction

        task run_phase(uvm_phase phase);
            super.run_phase(phase);
            forever begin
                sb_fifo.get(seq_item_sb);
                ref_model(seq_item_sb);
                if (seq_item_sb.dataout != shift_reg_out_ref) begin
                    `uvm_error("run_phase", $sformatf("Comparsion failed, Transaction received by the DUT:%s  While the reference out:0b%0b
                error_count++;
                end
                else begin
                    `uvm_info("run_phase", $sformatf("Correct shift_reg out: %s ", seq_item_sb.convert2string()), UVM_HIGH);
                    correct_count++;
                end
            end
        endtask

        task ref_model(shift_reg_seq_item seq_item_chk);

                case (seq_item_chk.mode)
                    SHIFT:
                        if (seq_item_chk.direction == LEFT)
                            shift_reg_out_ref = {seq_item_chk.datain[4:0], seq_item_chk.serial_in};
                        else if (seq_item_chk.direction == RIGHT)
                            shift_reg_out_ref = {seq_item_chk.serial_in, seq_item_chk.datain[5:1]};
                    ROTATE:
                        if (seq_item_chk.direction == LEFT)
                            shift_reg_out_ref = {seq_item_chk.datain[4:0], seq_item_chk.datain[5]};
                        else if (seq_item_chk.direction == RIGHT)
                            shift_reg_out_ref = {seq_item_chk.datain[0], seq_item_chk.datain[5:1]};
                endcase
        endtask

        function void report_phase(uvm_phase phase);
            super.report_phase(phase);
            `uvm_info("report_phase", $sformatf("Total successful transactions: %0d",correct_count), UVM_LOW);
            `uvm_info("report_phase", $sformatf("Total failed transactions: %0d",error_count), UVM_LOW);
        endfunction

    endclass
endpackage : shift_reg_scoreboard_pkg
```

29. Shift reg seq item:

```
package shift_reg_seq_item_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    import shared_pkg::*;

    class shift_reg_seq_item extends uvm_sequence_item;
        `uvm_object_utils(shift_reg_seq_item)

          rand bit serial_in;
          rand direction_e direction;
          rand bit mode;
          rand bit [5:0] datain;
        logic [5:0] dataout;

        function new(string name = "shift_reg_seq_item");
            super.new(name);
        endfunction

        function string convert2string();
            return $sformatf("%s  serial_in = %0xb, direction = %s, mode = %s, datain = %0xb", super.convert2string(),  serial_in, directio
        endfunction

        function string convert2string_stimulus();
            return $sformatf(" serial_in = %0xb, direction = %s, mode = %s, datain = %0xb", serial_in, direction.name(), mode, datain);
        endfunction
    endclass


endpackage
```

30. Shift reg sqr:

```
package shift_reg_sequencer_pkg;

    import uvm_pkg::*;
    `include "uvm_macros.svh"

    import shift_reg_seq_item_pkg::*;
    class shift_reg_sequencer extends uvm_sequencer #(shift_reg_seq_item);
        `uvm_component_utils(shift_reg_sequencer);

        function new (string name = "shift_reg_sequencer", uvm_component parent = null);
            super.new(name, parent);
        endfunction
    endclass

endpackage
```

31. Src file:

```
shift_reg_config_pkg.sv
alsu_config_pkg.sv
shared_pkg_alsu.sv
shift_reg_if.sv
alsu_if.sv
shift_reg.sv
ALSU.sv
shift_reg_seq_item_pkg.sv
shift_reg_sequencer_pkg.sv
shift_reg_driver_pkg.sv
shift_reg_monitor_pkg.sv
shift_reg_agent_pkg.sv
shift_reg_coverage_pkg.sv
shift_reg_scoreboard_pkg.sv
shift_reg_env_pkg.sv
shift_reg_main_seq_pkg.sv
golden_design.sv
alsu_seq_item_pkg.sv
alsu_sequencer_pkg.sv
alsu_driver_pkg.sv
alsu_monitor_pkg.sv
alsu_agent_pkg.sv
alsu_coverage_pkg.sv
alsu_scoreboard_pkg.sv
alsu_env_pkg.sv
alsu_reset_seq_pkg.sv
alsu_main_seq_pkg.sv
alsu_test_pkg.svq
alsu_sva.sv
alsu_top.sv
```

32. Coverage text of alsu:

```
Directive Coverage:
    Directives                    19        19         0   100.00%

DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------------
Name                                        Design Design   Lang File(Line)       Hits Status
                                            Unit   UnitType
--------------------------------------------------------------------------------------------
/alsu_top/dut/sva_inst/c1                   alsu_sva Verilog  SVA  alsu_sva.sv(6)     92 Covered
/alsu_top/dut/sva_inst/c2                   alsu_sva Verilog  SVA  alsu_sva.sv(11) 2004 Covered
/alsu_top/dut/sva_inst/c3                   alsu_sva Verilog  SVA  alsu_sva.sv(16) 1822 Covered
/alsu_top/dut/sva_inst/c4                   alsu_sva Verilog  SVA  alsu_sva.sv(23) 24180 Covered
/alsu_top/dut/sva_inst/c5                   alsu_sva Verilog  SVA  alsu_sva.sv(28) 1608 Covered
/alsu_top/dut/sva_inst/c6                   alsu_sva Verilog  SVA  alsu_sva.sv(33) 3364 Covered
/alsu_top/dut/sva_inst/c7                   alsu_sva Verilog  SVA  alsu_sva.sv(38) 3244 Covered
/alsu_top/dut/sva_inst/c8                   alsu_sva Verilog  SVA  alsu_sva.sv(43) 2050 Covered
/alsu_top/dut/sva_inst/c9                   alsu_sva Verilog  SVA  alsu_sva.sv(48)  580 Covered
/alsu_top/dut/sva_inst/c10                  alsu_sva Verilog  SVA  alsu_sva.sv(53) 1200 Covered
/alsu_top/dut/sva_inst/c11                  alsu_sva Verilog  SVA  alsu_sva.sv(58)  988 Covered
/alsu_top/dut/sva_inst/c12                  alsu_sva Verilog  SVA  alsu_sva.sv(63) 2004 Covered
/alsu_top/dut/sva_inst/c13                  alsu_sva Verilog  SVA  alsu_sva.sv(68) 2012 Covered
/alsu_top/dut/sva_inst/c14                  alsu_sva Verilog  SVA  alsu_sva.sv(73) 2014 Covered
/alsu_top/dut/sva_inst/c15                  alsu_sva Verilog  SVA  alsu_sva.sv(78)  958 Covered
/alsu_top/dut/sva_inst/c16                  alsu_sva Verilog  SVA  alsu_sva.sv(84) 1054 Covered
/alsu_top/dut/sva_inst/c17                  alsu_sva Verilog  SVA  alsu_sva.sv(89)  954 Covered
/alsu_top/dut/sva_inst/c18                  alsu_sva Verilog  SVA  alsu_sva.sv(94) 1054 Covered
/alsu_top/dut/sva_inst/c19                  alsu_sva Verilog  SVA  alsu_sva.sv(100)26910 Covered
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         38        38         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /alsu_top/dut --

                                              Node      1H->0L      0L->1H                          "Coverage"
                                              ---------------------------------------------------------------
                                        A_reg[2-0]           1           1                              100.00
                                        B_reg[2-0]           1           1                              100.00
                                      bypass_A_reg           1           1                              100.00
                                      bypass_B_reg           1           1                              100.00
                                        cin_reg[0]           1           1                              100.00
                                     direction_reg           1           1                              100.00
                                           invalid           1           1                              100.00
                                    invalid_opcode           1           1                              100.00
                                    invalid_red_op           1           1                              100.00
                                   opcode_reg[2-0]           1           1                              100.00
                                      red_op_A_reg           1           1                              100.00
                                      red_op_B_reg           1           1                              100.00
                                     serial_in_reg           1           1                              100.00

Total Node Count     =         19
Toggled Node Count   =         19
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (38 of 38 bins)
```

```
Assertion Coverage:
    Assertions                      20        20         0   100.00%
--------------------------------------------------------------------
Name                 File(Line)                   Failure      Pass
                                                  Count        Count
--------------------------------------------------------------------
/alsu_top/dut/sva_inst/a1
                     alsu_sva.sv(4)                     0          1
/alsu_top/dut/sva_inst/a2
                     alsu_sva.sv(9)                     0          1
/alsu_top/dut/sva_inst/a3
                     alsu_sva.sv(14)                    0          1
/alsu_top/dut/sva_inst/a4
                     alsu_sva.sv(20)                    0          1
/alsu_top/dut/sva_inst/a5
                     alsu_sva.sv(26)                    0          1
/alsu_top/dut/sva_inst/a6
                     alsu_sva.sv(31)                    0          1
/alsu_top/dut/sva_inst/a7
                     alsu_sva.sv(36)                    0          1
/alsu_top/dut/sva_inst/a8
                     alsu_sva.sv(41)                    0          1
/alsu_top/dut/sva_inst/a9
                     alsu_sva.sv(46)                    0          1
/alsu_top/dut/sva_inst/a10
                     alsu_sva.sv(51)                    0          1
/alsu_top/dut/sva_inst/a11
                     alsu_sva.sv(56)                    0          1
/alsu_top/dut/sva_inst/a12
                     alsu_sva.sv(61)                    0          1
/alsu_top/dut/sva_inst/a13
                     alsu_sva.sv(66)                    0          1
/alsu_top/dut/sva_inst/a14
                     alsu_sva.sv(71)                    0          1
/alsu_top/dut/sva_inst/a15
                     alsu_sva.sv(76)                    0          1
/alsu_top/dut/sva_inst/a16
                     alsu_sva.sv(82)                    0          1
/alsu_top/dut/sva_inst/a17
                     alsu_sva.sv(87)                    0          1
/alsu_top/dut/sva_inst/a18
                     alsu_sva.sv(92)                    0          1
/alsu_top/dut/sva_inst/a19
                     alsu_sva.sv(97)                    0          1
/alsu_top/dut/sva_inst/o_1
                     alsu_sva.sv(105)                   0          1
```

```
=== Instance: /alsu_top/dut
=== Design Unit: work.ALSU
=========================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        32        31         1    96.87%

==============================Branch Details==============================

Branch Coverage for instance /alsu_top/dut

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File ALSU.sv
------------------------------------IF Branch------------------------------------
    15                                     51959     Count coming in to IF
    15              1                        407       if(a_if.rst) begin

    26              1                      51552       end else begin

Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    42                                     52204     Count coming in to IF
    42              1                        612       if(a_if.rst) begin

    44              1                      51592       end else begin

Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    45                                     51592     Count coming in to IF
    45              1                      27057           if (invalid)

    47              1                      24535           else

Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    54                                     52204     Count coming in to IF
    54              1                        612       if(a_if.rst) begin

    57              1                      51592       else begin

Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    58                                     51592     Count coming in to IF
    58              1                         92         if (bypass_A_reg && bypass_B_reg)

    60              1                       2013         else if (bypass_A_reg)

    62              1                       1830         else if (bypass_B_reg)

Branch totals: 5 hits of 5 branches = 100.00%

------------------------------------CASE Branch------------------------------------
    67                                     23346     Count coming in to CASE
    68              1                      10510               3'h0: begin

    78              1                       4788               3'h1: begin

    88              1                       2013               3'h2: begin

    94              1                       2014               3'h3: a_if.out <= A_reg * B_reg;

    95              1                       2012               3'h4: begin

    101             1                       2009               3'h5: begin

                                         ***0***     All False Count
Branch totals: 6 hits of 7 branches = 85.71%

------------------------------------IF Branch------------------------------------
    69                                     10510     Count coming in to IF
    69              1                       1615                 if (red_op_A_reg && red_op_B_reg)

    71              1                       3264                 else if (red_op_A_reg)

    73              1                       3379                 else if (red_op_B_reg)

    75              1                       2252                 else

Branch totals: 4 hits of 4 branches = 100.00%

------------------------------------IF Branch------------------------------------
    79                                      4788     Count coming in to IF
    79              1                        580                 if (red_op_A_reg && red_op_B_reg)

    81              1                        998                 else if (red_op_A_reg)
```

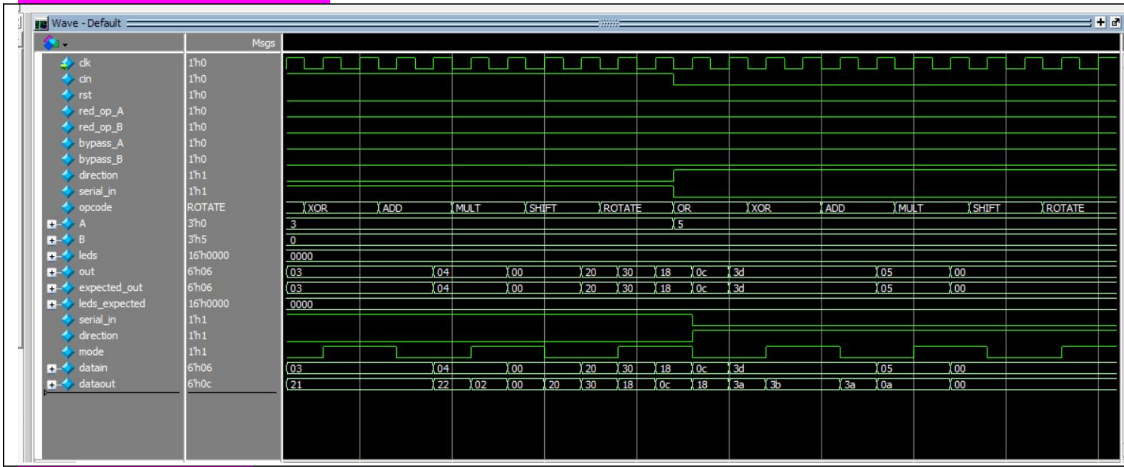
Branch coverage and total coverage are not 100% because there is no default case. Instead, a **false count** statement exists. When the opcode takes an default value, the false count becomes one. However, the default opcode values 6 and 7 are already handled separately as invalid, outside the case statement. Therefore, when the opcode is 6 or 7, the default case is never reached, and the false count will never be triggered.

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      48        48         0   100.00%

================================Statement Details================================

Statement Coverage for instance /alsu_top/dut --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File ALSU.sv
    3                                                module ALSU(alsu_if.DUT a_if);

    4                                                reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;

    5                                                reg  [1:0] cin_reg;

    6                                                reg [2:0] opcode_reg;

    7                                                reg signed [2:0] A_reg, B_reg;

    8                                                wire invalid_red_op, invalid_opcode, invalid;

    9                                                //Invalid handling

    10              1                      23924     assign invalid_red_op = (red_op_A_reg || red_op_B_reg) & (opcode_reg[1] || opcode_reg[2]);
```

```
Covergroup Coverage:
    Covergroups                      1        na        na   100.00%
        Coverpoints/Crosses         18        na        na        na
            Covergroup Bins         46        46         0   100.00%
----------------------------------------------------------------------------------------------------------
Covergroup                                          Metric       Goal       Bins    Status

----------------------------------------------------------------------------------------------------------
 TYPE /alsu_coverage_pkg/alsu_coverage/alsu_cg       100.00%        100          -    Covered
    covered/total bins:                                  46         46          -
    missing/total bins:                                   0         46          -
    % Hit:                                          100.00%        100          -
    Coverpoint A_cp1                                100.00%        100          -    Covered
        covered/total bins:                               3          3          -
        missing/total bins:                               0          3          -
        % Hit:                                      100.00%        100          -
    Coverpoint A_cp2                                100.00%        100          -    Covered
        covered/total bins:                               2          2          -
        missing/total bins:                               0          2          -
        % Hit:                                      100.00%        100          -
    Coverpoint B_cp1                                100.00%        100          -    Covered
        covered/total bins:                               3          3          -
        missing/total bins:                               0          3          -
        % Hit:                                      100.00%        100          -
    Coverpoint B_cp2                                100.00%        100          -    Covered
        covered/total bins:                               2          2          -
        missing/total bins:                               0          2          -
        % Hit:                                      100.00%        100          -
    Coverpoint ALU_cp                               100.00%        100          -    Covered
        covered/total bins:                               8          8          -
        missing/total bins:                               0          8          -
        % Hit:                                      100.00%        100          -
    Coverpoint c_cp                                 100.00%        100          -    Covered
        covered/total bins:                               2          2          -
        missing/total bins:                               0          2          -
        % Hit:                                      100.00%        100          -
    Coverpoint direction_cp                         100.00%        100          -    Covered
        covered/total bins:                               2          2          -
        missing/total bins:                               0          2          -
        % Hit:                                      100.00%        100          -
    Coverpoint serial_cp                            100.00%        100          -    Covered
        covered/total bins:                               2          2          -
        missing/total bins:                               0          2          -
        % Hit:                                      100.00%        100          -
    Coverpoint reda_cp                              100.00%        100          -    Covered
        covered/total bins:                               2          2          -
        missing/total bins:                               0          2          -
        % Hit:                                      100.00%        100          -
    Coverpoint redb_cp                              100.00%        100          -    Covered
        covered/total bins:                               2          2          -
        missing/total bins:                               0          2          -
        % Hit:                                      100.00%        100          -
    Cross c1_cs                                     100.00%        100          -    Covered
        covered/total bins:                               3          3          -
```

```
            % Hit:                                       100.00%       100          -
            Auto, Default and User Defined Bins:
                bin add_mult_b1                             2432         1          -     Covered
                bin add_mult_b2                             1915         1          -     Covered
                bin add_mult_b3                             1418         1          -     Covered
        Cross c2_cs                                      100.00%       100          -     Covered
            covered/total bins:                                3         3          -
            missing/total bins:                                0         3          -
            % Hit:                                       100.00%       100          -
            Auto, Default and User Defined Bins:
                bin add_mult_a1                             2445         1          -     Covered
                bin add_mult_a2                             1998         1          -     Covered
                bin add_mult_a3                             1402         1          -     Covered
        Cross c3_cs                                      100.00%       100          -     Covered
            covered/total bins:                                1         1          -
            missing/total bins:                                0         1          -
            % Hit:                                       100.00%       100          -
            Auto, Default and User Defined Bins:
                bin add                                     3765         1          -     Covered
        Cross c4_cs                                      100.00%       100          -     Covered
            covered/total bins:                                2         2          -
            missing/total bins:                                0         2          -
            % Hit:                                       100.00%       100          -
            Auto, Default and User Defined Bins:
                bin sh                                      3713         1          -     Covered
                bin r                                       3708         1          -     Covered
        Cross c5_cs                                      100.00%       100          -     Covered
            covered/total bins:                                1         1          -
            missing/total bins:                                0         1          -
            % Hit:                                       100.00%       100          -
            Auto, Default and User Defined Bins:
                bin sh                                      3713         1          -     Covered
        Cross c6_cs                                      100.00%       100          -     Covered
            covered/total bins:                                1         1          -
            missing/total bins:                                0         1          -
            % Hit:                                       100.00%       100          -
            Auto, Default and User Defined Bins:
                bin reda                                    3247         1          -     Covered
        Cross c7_cs                                      100.00%       100          -     Covered
            covered/total bins:                                1         1          -
            missing/total bins:                                0         1          -
            % Hit:                                       100.00%       100          -
            Auto, Default and User Defined Bins:
                bin redb                                    3404         1          -     Covered
        Cross c8_cs                                      100.00%       100          -     Covered
            covered/total bins:                                6         6          -
            missing/total bins:                                0         6          -
            % Hit:                                       100.00%       100          -
            Auto, Default and User Defined Bins:
                bin a1                                       217         1          -     Covered
                bin a2                                       170         1          -     Covered
                bin b1                                       217         1          -     Covered
                bin b2                                       210         1          -     Covered
                bin ab1                                     4976         1          -     Covered
                bin ab2                                     5209         1          -     Covered
```

33. Wave form :



34. Output :

```
,leds=0000000000000000
# UVM_INFO alsu_scoreboard_pkg.sv(39) @ 104004: uvm_test_top.a_env.score_b [run_phase] expected_out value : 000110 , leds_expected : 0000000000000000
# UVM_INFO alsu_test_pkg.sv(61) @ 104004: uvm_test_top [run_phase] Stimulus Generation Ended
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 104004: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO alsu_scoreboard_pkg.sv(54) @ 104004: uvm_test_top.a_env.score_b [report_phase] Total successful transactions: 26001
# UVM_INFO alsu_scoreboard_pkg.sv(55) @ 104004: uvm_test_top.a_env.score_b [report_phase] Total failed transactions: 0
# UVM_INFO shift_reg_scoreboard_pkg.sv(68) @ 104004: uvm_test_top.s_env.sb [report_phase] Total successful transactions: 52002
# UVM_INFO shift_reg_scoreboard_pkg.sv(69) @ 104004: uvm_test_top.s_env.sb [report_phase] Total failed transactions: 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :52014
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [TEST_DONE]     1
# [report_phase]     4
# [run_phase] 52006
# ** Note: $finish    : C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 104004 ns  Iteration: 61  Instance: /alsu_top
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

35. Do file:

```
vlib work
vlog -f src_files.list.txt +cover -covercells
vsim   -voptargs=+acc work.alsu_top -classdebug -uvmcontrol=all -cover
add wave /alsu_top/a_if/*
add wave /alsu_top/shiftregif/*
coverage save alsu_top.ucdb -onexit
run -all
coverage exclude -du ALSU -togglenode {cin_reg[1]}
```

36. Shift reg coverage text:

```
=== Instance: /alsu_top/DUT
=== Design Unit: work.shift_reg
=====================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         5         5         0   100.00%

================================Branch Details================================

Branch Coverage for instance /alsu_top/DUT

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File shift_reg.sv
------------------------------------IF Branch------------------------------------
    13                                     50205     Count coming in to IF
    13              1                      21373           if (shiftregif.mode) // rotate

    19              1                      14323              if (shiftregif.direction) // left

    21              1                      14509              else

Branch totals: 3 hits of 3 branches = 100.00%

------------------------------------IF Branch------------------------------------
    14                                     21373     Count coming in to IF
    14              1                      10694              if (shiftregif.direction) // left

    16              1                      10679              else

Branch totals: 2 hits of 2 branches = 100.00%


Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       5         5         0   100.00%
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       5         5         0   100.00%

================================Statement Details================================

Statement Coverage for instance /alsu_top/DUT --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File shift_reg.sv
    8                                                module shift_reg (shift_reg_if.DUT shiftregif);

    9

    10

    11             1                           1     always @(*) begin

    12

    13                                                     if (shiftregif.mode) // rotate

    14                                                        if (shiftregif.direction) // left

    15             1                       10694                 shiftregif.dataout = {shiftregif.datain[4:0], shiftregif.datain[5]};

    16                                                        else

    17             1                       10679                 shiftregif.dataout = {shiftregif.datain[0], shiftregif.datain[5:1]};

    18                                                     else // shift

    19                                                        if (shiftregif.direction) // left

    20             1                       14323                 shiftregif.dataout = {shiftregif.datain[4:0], shiftregif.serial_in};

    21                                                        else

    22             1                       14509                 shiftregif.dataout = {shiftregif.serial_in, shiftregif.datain[5:1]};



================================================================================
=== Instance: /alsu_top
=== Design Unit: work.alsu_top
```

```
================================================================================
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         30        30         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /alsu_top/shiftregif --

                                              Node      1H->0L      0L->1H                          "Coverage"
                                              ---------------------------------------------------------------
                                         datain[5-0]           1           1                              100.00
                                        dataout[5-0]           1           1                              100.00
                                           direction           1           1                              100.00
                                                mode           1           1                              100.00
                                           serial_in           1           1                              100.00

Total Node Count     =         15
Toggled Node Count   =         15
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (30 of 30 bins)
```

```
=============================================================================

Covergroup Coverage:
    Covergroups                    1         na         na    98.43%
        Coverpoints/Crosses        4         na         na        na
            Covergroup Bins       70         66          4    94.28%
---------------------------------------------------------------------------------------------------
Covergroup                                             Metric       Goal       Bins    Status

---------------------------------------------------------------------------------------------------
 TYPE /shift_reg_coverage_pkg/shift_reg_coverage/cvr_grp
                                                       98.43%        100          -    Uncovered
    covered/total bins:                                    66         70          -
    missing/total bins:                                     4         70          -
    % Hit:                                             94.28%        100          -
    Coverpoint datain_cvp                              93.75%        100          -    Uncovered
        covered/total bins:                                60         64          -
        missing/total bins:                                 4         64          -
        % Hit:                                         93.75%        100          -
        bin auto[0]                                     44743          1          -    Covered
        bin auto[1]                                     13739          1          -    Covered
        bin auto[2]                                      1119          1          -    Covered
        bin auto[3]                                      1915          1          -    Covered
        bin auto[4]                                       306          1          -    Covered
        bin auto[5]                                       202          1          -    Covered
        bin auto[6]                                       443          1          -    Covered
        bin auto[7]                                       308          1          -    Covered
        bin auto[8]                                       334          1          -    Covered
        bin auto[9]                                       263          1          -    Covered
        bin auto[10]                                      152          1          -    Covered
        bin auto[11]                                      150          1          -    Covered
        bin auto[12]                                      356          1          -    Covered
        bin auto[13]                                      149          1          -    Covered
        bin auto[14]                                       86          1          -    Covered
        bin auto[15]                                      114          1          -    Covered
        bin auto[16]                                      152          1          -    Covered
        bin auto[17]                                       19          1          -    Covered
        bin auto[18]                                       10          1          -    Covered
        bin auto[19]                                       72          1          -    Covered
        bin auto[20]                                        0          1          -    ZERO
        bin auto[21]                                        0          1          -    ZERO
        bin auto[22]                                       16          1          -    Covered
        bin auto[23]                                       13          1          -    Covered
        bin auto[24]                                      128          1          -    Covered
        bin auto[25]                                       19          1          -    Covered
        bin auto[26]                                       19          1          -    Covered
        bin auto[27]                                       26          1          -    Covered
        bin auto[28]                                       51          1          -    Covered
        bin auto[29]                                       18          1          -    Covered
        bin auto[30]                                       44          1          -    Covered
        bin auto[31]                                       88          1          -    Covered
        bin auto[32]                                      194          1          -    Covered
```

There is some bins from the auto generated bins did not hit